AFL Fantasy Assistant on AWS: solution architecture

Helps each coach build a starting squad, keep versions as news arrives, lock it for round 1, then get weekly trade advice and plan a draft. Every coach sees only their own data.

1. COACHES

Coach A / Coach B
Web app in the browser (React). Sign in, build squads, ask questions, review advice.

Their choices
Squad versions, age cap, avoid list, premium locks, league settings, injury or doubt notes.

→

2. IDENTITY AND ACCESS

Amazon Cognito
Sign-in. Issues a signed token that says who the coach is.

AgentCore Gateway + Cedar policy
Checks the token on every tool call. Default deny. Rules sit outside the AI model.

→

3. ASSISTANT

AgentCore Runtime (agent)
Understands the question, picks tools, explains results in plain language. Uses a foundation model on Amazon Bedrock.

What it never does
Decide access, change projections, or execute trades. The coach decides.

→

4. TOOLS AND ENGINES (AWS)

Projection engine (core)
Projects every player for AFL Fantasy and SuperCoach, applies the developer's overrides, keeps one file per season. Looks ahead 3 to 4 rounds once the fixture exists.

Shortlist
The 50 to 60 players of interest. Built before any squad.

Squad builder + versions
Optimiser on the shortlist plus the coach's locks, under the cap. Explains its choices. Named versions; locks at round 1.

Weekly trade advisor
Trade options within the round limit, with the numbers behind each.

Draft planner
Rankings by position scaled to league size; pick plan.

Projections, preferences, expert views
The coach's rules; expert opinion with source and date.

→

5. DATA

Amazon S3 data lake (existing)
Player stats, 2026 player list, identity table. Catalogued by AWS Glue, queryable with Athena.

Projections and lineups (new)
One projection file per season, shortlists, fixtures, lineups, prices.

DynamoDB (new)
Each coach's squads, versions, preferences, leagues.

Expert knowledge (new)
Summaries of expert emails, searchable.

DEVELOPER SIDE (OWN MACHINE)

Existing local app
Scrapes fixtures, lineups and prices; keeps the model code.

→

Loader
Copies the results into S3 after each round.

→

Email intake
Reads expert emails from the developer's own Gmail; uploads summaries only.

Pre-season
Build squad; keep versions as news arrives

Round 0
Actual scores feed the next build

Lineup announced
Named players update the squad

Round 1: squad locks
Build mode becomes read-only

Every week
Trade advice only; injuries, form, role changes, news

Security

- Coach A cannot read Coach B's squads or preferences
- The coach's identity comes from the token, never from the AI
- No passwords or keys in code or notes

Trust

- One projected score per player, shared by every coach and the same every time
- The AI explains; it does not change the numbers
- Expert views are shown with source and date, beside the numbers

Cost

- Runs inside the free-tier account and credits (about $158 on 2026-10-10)
- Budget alarm before first use
- Cost measured after the build, by service and by use (counts of calls)

Not in this version

- Keeper and Dynasty drafts
- Logging in to the games or making trades
- Daily fantasy (DFS)

Draft v0.1 for discussion. Learning project in a personal AWS account, built with Claude (Claude desktop app). The data layer was checked against the S3, Glue and Lambda listings on 2026-10-10 (one bucket, 7.6 MiB; 3 Glue crawlers, no Glue jobs; data for projections, lineups and per-user state is not in AWS yet). Final app runs on the owner's machine; this AWS version is for learning and cost measurement.